Aaditya Goel[1]

[1]Python Developer HCLTech

March 13, 2026

Abstract

Large language models are increasingly applied to software test generation, but existing approaches often exhibit non deterministic behavior and hallucinated user interface elements. This paper presents a deterministic multi agent architecture for requirement driven behavior driven development test synthesis. The system integrates structured retrieval, grammar constraints, and multi layer guardrails to ensure reproducible and context grounded outputs. Experimental results demonstrate improved consistency, reduced hallucinations, and stronger compliance with canonical test patterns compared to conventional stochastic generation methods.

OrionTest: An Agentic AI Driven Automation*

Aaditya Goel
Python Developer
HCLTech, Noida
Delhi, India
goelaaditya2003@gmail.com

***Abstract*—Large language models are increasingly applied to software test generation, but existing approaches often exhibit non deterministic behavior and hallucinated user interface elements. This paper presents a deterministic multi agent architecture for requirement driven behavior driven development test synthesis. The system integrates structured retrieval, grammar constraints, and multi layer guardrails to ensure reproducible and context grounded outputs. Experimental results demonstrate improved consistency, reduced hallucinations, and stronger compliance with canonical test patterns compared to conventional stochastic generation methods.**

***Index Terms*—test automation, behavior driven development, deterministic inference, retrieval augmented generation, multi agent systems**

I. Introduction

Software testing is a critical component of modern software development, yet the creation and maintenance of automated test cases remain time consuming and labor intensive [15], [16]. Manual test case design requires significant effort in translating requirements into structured scenarios, implementing step definitions, maintaining locators, and validating execution results. As systems evolve rapidly, maintaining consistency between requirements, user interfaces, and automated test artifacts becomes increasingly challenging [17].

Recent advances in large language models have introduced new possibilities for automated test generation [5], [21]. However, existing approaches often exhibit non deterministic behavior, hallucinated user interface elements, weak grounding to real system interfaces, and limited reproducibility [7]. Such limitations reduce trust in generated artifacts and restrict adoption in enterprise environments where auditability and consistency are essential.

To address these challenges, this paper presents OrionTest, a deterministic multi agent architecture for requirement driven behavior driven development test synthesis. The proposed system decomposes the automation workflow into specialized agents responsible for requirement extraction, feature generation, step definition synthesis, user interface discovery, execution validation, defect analysis, and reporting. A deterministic inference protocol is enforced through fixed decoding parameters and controlled model evaluation to guarantee reproducible outputs for identical inputs. In addition, a retrieval augmented generation mechanism injects canonical framework rules, step patterns, and runtime user interface discovery results to constrain generation within verified domain boundaries. The main contributions of this work are as follows:

- A deterministic multi agent architecture for end to end test automation.
- A structured retrieval mechanism that grounds generation in framework rules and live user interface discovery.
- A grammar constrained generation model with explicit refusal semantics to prevent invalid test artifacts.
- An execution integrated validation pipeline that links generated tests with runtime verification.

Experimental evaluation demonstrates improved reproducibility, reduced hallucinated elements, and stronger compliance with canonical test patterns compared to conventional stochastic generation approaches.

II. Related Work

A. Automated Test Case Generation

Automated test generation has been widely studied in software engineering research. Traditional approaches rely on rule based systems, symbolic execution, and operational abstraction techniques [3], [17] to generate test cases from source code or formal specifications. Classical foundations of software testing further emphasize systematic validation strategies and structured test design methodologies [15], [16]. While these methods provide structured outputs, they often require significant manual configuration and are limited in handling natural language requirements.

Recent advances in large language models have enabled natural language driven test case generation [4], [21]. These models can translate informal requirements into structured test artifacts. However, existing approaches frequently exhibit non deterministic behavior, generate hallucinated user interface elements, and lack strict grammar enforcement [5], [7]. Such limitations reduce reliability in enterprise environments where reproducibility and structural correctness are essential.

B. Retrieval Augmented Generation

Retrieval augmented generation has been proposed to improve contextual grounding in large language models [7], [8]. By retrieving relevant documents during inference, models can incorporate domain specific knowledge without retraining. Dense retrieval and embedding based similarity mechanisms have also been explored for semantic ranking [23]. While effective for semantic tasks, embedding based retrieval introduces additional complexity and potential non determinism.

In structured domains such as behavior driven development, canonical patterns and framework rules are often more

effectively retrieved through deterministic lexical matching, consistent with classical information retrieval formulations [14]. This motivates the use of controlled retrieval mechanisms that prioritize reproducibility and auditability.

C. Multi Agent Language Model Systems

Multi agent architectures have emerged as a strategy for decomposing complex tasks into specialized components [9]. Instead of relying on a single monolithic prompt, agent based systems assign specific responsibilities to different modules, such as planning, execution, validation, and correction. Recent language agent frameworks further explore iterative reasoning and task refinement strategies [24], [25]. This decomposition improves modularity and traceability.

However, many existing multi agent systems focus primarily on task completion rather than deterministic behavior or governance controls. The integration of strict grammar constraints, reproducible decoding, and structured refusal policies remains limited in current literature.

D. Deterministic and Trustworthy AI Systems

Trustworthy artificial intelligence systems require reproducibility, transparency, and controlled behavior [10]. In high assurance domains, deterministic inference and explicit validation layers are essential for maintaining reliability. Software testing standards further emphasize documentation, validation traceability, and structured verification processes [13], [27]. While several studies address fairness and explainability, fewer works focus on enforcing deterministic decoding and grammar constrained generation in applied software engineering workflows.

This paper extends existing research by combining deterministic inference, structured retrieval grounding, multi agent orchestration, and execution level validation into a unified automation framework.

III. Problem Formulation

Let R denote a set of raw functional requirements expressed in natural language. Let U represent user interface discovery outputs collected from the target system at runtime. Let S denote structured framework knowledge, including canonical grammar rules, step definition patterns, and domain specific documentation.

The objective of the system is to generate executable behavior driven development test artifacts T that are syntactically valid, context grounded, and reproducible. Formally, we define a transformation function:

$$T = F(R, U, S) \tag{1}$$

where F represents the end to end automation pipeline composed of retrieval augmented grounding and deterministic language modeling [7], [8].

A. Retrieval Grounded Context Construction

Given a query derived from requirements R, relevant documents are retrieved from the corpus S using a deterministic ranking mechanism. The ranking strategy follows classical term frequency inverse document frequency weighting from information retrieval theory [14].

For a term t in document d, term frequency is defined as:

$$TF(t, d) = 1 + \log(f_{t,d}) \tag{2}$$

where $f_{t,d}$ denotes the frequency of term t in document d. Inverse document frequency is defined as:

$$IDF(t) = \log\left(\frac{N+1}{df(t)+1}\right) + 1 \tag{3}$$

where N is the total number of documents and $df(t)$ is the number of documents containing term t.

The relevance score between query q and document d is computed as:

$$Score(q, d) = \sum_{t \in q \cap d} TF(t, d) \cdot TF(t, q) \cdot IDF(t) \tag{4}$$

The top ranked documents are injected into the model context to construct a grounded prompt, consistent with retrieval augmented generation paradigms [7], [8].

B. Deterministic Generation Objective

Let M denote the language model with fixed parameters. The underlying autoregressive formulation follows the transformer architecture [11]. For a given input context x, token generation at time step t follows:

$$y_t = \arg\max_{w \in V} P(w \mid y_{<t}, x) \tag{5}$$

where V is the vocabulary and sampling is disabled. This greedy decoding ensures deterministic behavior such that identical inputs produce identical outputs.

C. Constrained Output Validation

A validation function G enforces structural constraints on generated output T. The final artifact is defined as:

$$T^* = \begin{cases} T, & \text{if constraints are satisfied} \\ \text{error}, & \text{otherwise} \end{cases} \tag{6}$$

The overall system therefore operates as a composed deterministic mapping from requirements to executable test artifacts under retrieval grounding and structural validation. This composition can be interpreted within modular multi agent system formulations [9].

IV. Architecture Design

This section presents the high level architectural design of OrionTest. The framework is constructed as a deterministic multi agent pipeline that transforms natural language requirements into validated and executable behavior driven development artifacts. The architecture emphasizes modular decomposition, retrieval grounded generation, constraint enforcement, and execution level validation.

A. Design Objectives

The architecture is guided by four primary design principles:

- **Determinism:** Identical inputs must produce identical outputs under fixed model parameters.
- **Grounded Generation:** All generated artifacts must be constrained by framework rules and runtime user interface context.
- **Modularity:** Each transformation stage is handled by a specialized agent to ensure traceability.
- **Validation First:** Generated artifacts must pass structural and grammar constraints before execution.

These objectives ensure reproducibility, reliability, and enterprise readiness.

B. Layered Architectural View

The system is organized into five logical layers:

1) **Orchestration Layer**
2) **Agent Processing Layer**
3) **Retrieval and Context Construction Layer**
4) **Deterministic Inference Layer**
5) **Execution and Reporting Layer**

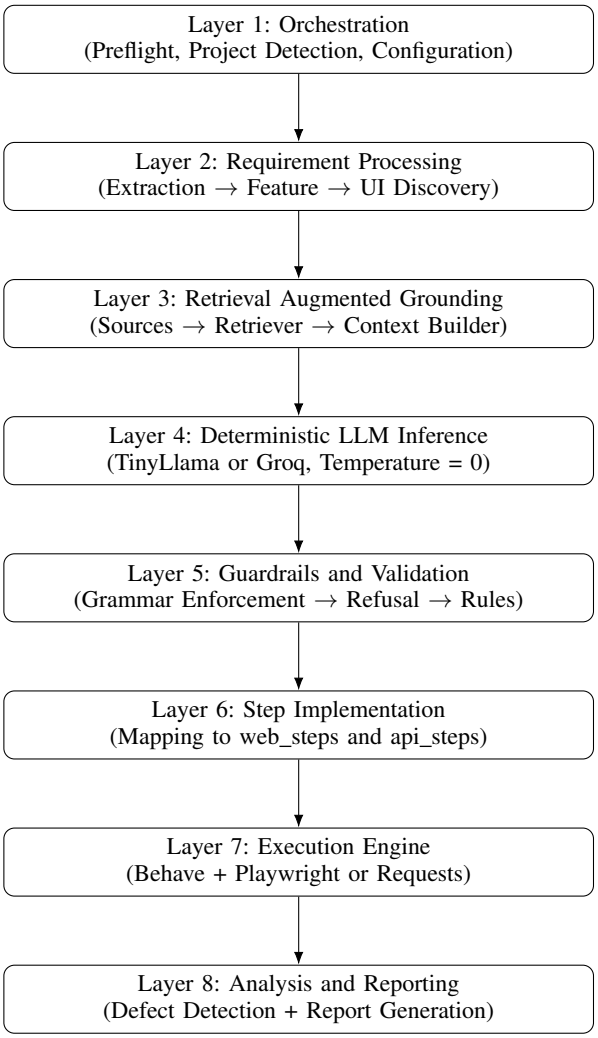

Fig. 1. Layered architecture of OrionTest showing orchestration, agent processing, retrieval grounding, deterministic inference, and execution validation.

C. Orchestration Layer

The orchestration layer coordinates the end to end workflow and manages inter agent communication. This hierarchical coordination mechanism is consistent with established multi agent system architectures [9]. It manages execution order, validates environment readiness, and routes tasks based on project type. Configuration management and environment initialization are handled at this stage to ensure reproducibility and controlled execution.

D. Agent Processing Layer

The agent processing layer decomposes the overall transformation task into specialized modules, following modular multi agent design principles [9]. Each agent performs a deterministic transformation from one structured representation to another. Requirement normalization, feature generation, step synthesis, and user interface discovery are handled independently. This modular design improves traceability and simplifies validation in complex automation pipelines.

E. Retrieval and Context Construction Layer

Prior to generation, the retrieval layer collects relevant domain knowledge using retrieval augmented generation principles [7], [8]. Canonical grammar rules, framework patterns, example features, runtime user interface discovery outputs, and locator metadata are ranked and formatted into a structured context. The ranking strategy aligns with classical information retrieval models [14]. This context is injected into the model input to restrict generation within verified domain boundaries.

F. Deterministic Inference Layer

The deterministic inference layer executes the language model under fixed parameters. The underlying autoregressive modeling follows the transformer architecture [11]. Sampling is disabled and greedy decoding is applied to guarantee reproducible outputs. Model weights remain static during inference, and no online learning is permitted. This layer ensures that the system behaves as a deterministic mapping from structured input to structured output.

G. Execution and Reporting Layer

Validated artifacts are executed within the integrated automation framework. Automated web testing infrastructures have been widely studied in prior work [12], and modern browser automation platforms such as Playwright provide robust cross browser control [28]. Browser lifecycle management, API invocation, timeout control, and error handling are managed at this stage. Execution results are analyzed by defect analysis components and summarized through reporting modules, completing the end to end automation loop in alignment with established testing standards [13], [27].

V. Agent Design and Responsibilities

The proposed framework adopts a modular multi agent design in which each agent performs a deterministic transformation within the automation pipeline, consistent with multi agent architectural paradigms [9]. This section describes the role and responsibilities of each agent.

A. Requirements Extraction Agent

The requirements extraction agent processes raw natural language requirements and converts them into structured representations. The normalization process aligns with structured requirement interpretation methodologies in software engineering research [17]. The transformation ensures that intent is explicitly captured before feature synthesis.

B. Requirements to Feature Agent

The requirements to feature agent converts structured intent into canonical behavior driven development feature files, following established BDD principles [1]. This agent enforces subject normalization, step keyword constraints, and present tense consistency. Grammar rules are applied during generation to ensure syntactic correctness and structural compliance.

C. Feature to Step Definition Agent

The feature to step definition agent translates generated feature steps into executable automation definitions. Step mapping follows predefined templates aligned with automation framework conventions and structured testing methodologies [15], [16]. Locator prioritization strategies are applied to ensure stable element identification.

D. Requirements Aware User Interface Discovery Agent

This agent bridges requirement intent with runtime system state. Based on the generated feature, it identifies relevant application pages and components to inspect. This alignment ensures that only contextually relevant interface elements are retrieved, supporting retrieval grounded generation principles [7].

E. Web Discovery Agent

The web discovery agent extracts user interface elements from the browser context. Element roles, attributes, identifiers, and accessible names are captured to support grounded generation and locator synthesis. Such browser level inspection aligns with automated web testing frameworks [12].

F. XPath Discovery Agent

The XPath discovery agent generates structured locator strategies for discovered elements. Multiple locator priorities are considered to ensure robustness and minimize brittle test behavior, consistent with best practices in automated testing literature [16].

G. Execution Agent

The execution agent triggers the behavior driven development test framework and manages runtime execution. Browser lifecycle, API calls, timeouts, and context handling are controlled at this stage in alignment with modern automation infrastructure practices [28].

H. Defect Agent

Upon execution failure, the defect agent analyzes error traces and classifies issues based on runtime context. Structured defect summaries are generated to assist debugging, consistent with structured testing documentation standards [13].

I. Reporting Agent

The reporting agent aggregates execution outcomes and produces structured reports. These reports summarize pass fail statistics, failure reasons, and contextual metadata to support traceability and governance requirements [27].

J. Agent Interaction Model

Agents are invoked sequentially through the orchestration layer, forming a deterministic transformation chain consistent with multi agent coordination models [9]. Each agent receives validated input from the previous stage and produces structured output for the next stage. This deterministic transformation chain enhances traceability and simplifies debugging.

The modular separation of responsibilities ensures that individual components can be updated without affecting the overall architecture, while preserving deterministic system behavior and governance compliance [10].

VI. Deterministic Language Model Adaptation

This section describes the adaptation and deterministic deployment of the language model used within the proposed framework.

A. Base Model Selection

The system employs a lightweight open source transformer based language model as the base architecture, following the transformer formulation introduced in prior work [11]. Compact transformer models have demonstrated strong instruction following capabilities while enabling efficient deployment [5], [21]. A compact model was selected to enable local deployment, reproducibility, and cost efficiency. The model supports instruction style generation and is suitable for domain specific fine tuning.

B. Domain Specific Fine Tuning

To improve alignment with behavior driven development patterns and automation conventions, the base model was fine tuned using a parameter efficient adaptation strategy. Specifically, a low rank adaptation technique was applied to inject domain specific knowledge without modifying the full set of model parameters [6].

The training dataset consisted of approximately 2000 structured samples, including canonical feature files, step definitions, and framework compliant automation examples. During validation, the adapted model achieved approximately 85 percent token level accuracy. This indicates strong structural alignment with domain specific patterns while maintaining the compact footprint of the base model.

The fine tuning process improved structural consistency and reduced off domain generation behavior while preserving the general language modeling capability of the pretrained transformer [11].

C. Adapter Integration

The fine tuned adapter is loaded during model initialization and combined with the frozen base model. This parameter efficient adaptation mechanism preserves the original pretrained knowledge while incorporating task specific specialization [6]. The adapter parameters remain fixed during inference to ensure consistent behavior.

D. Deterministic Inference Configuration

The inference pipeline enforces deterministic decoding by disabling sampling and applying greedy token selection. The underlying autoregressive generation process follows transformer based probabilistic modeling [11]. For each decoding step, the token with maximum probability is selected. No temperature scaling or random sampling is applied.

This configuration ensures that identical inputs produce identical outputs under fixed model weights. As a result, the generation process behaves as a deterministic mapping rather than a stochastic sampling process, addressing reliability concerns associated with unconstrained large language model deployment [5].

E. Model Freeze Policy

The deployed system operates under a strict model freeze policy. Model weights, adapter parameters, and retrieval corpora remain static during inference. No online learning or dynamic weight updates are permitted. This governance aligned configuration supports reproducibility and auditability requirements emphasized in trustworthy artificial intelligence systems [10].

F. Reproducibility Guarantee

Given fixed input context, static retrieval sources, and deterministic decoding parameters, the system produces identical outputs across repeated runs. This reproducibility property is critical for enterprise adoption where traceability and consistency are required.

VII. Retrieval Augmented Grounding Mechanism

To reduce hallucinated outputs and improve domain alignment, the proposed framework integrates a deterministic retrieval augmented grounding mechanism consistent with retrieval augmented generation paradigms [7], [8]. Unlike embedding based retrieval systems that rely on dense similarity models [23], the proposed approach relies on structured lexical ranking and controlled corpus construction to ensure reproducibility.

A. Knowledge Sources

The retrieval corpus consists of multiple structured sources grounded in behavior driven development conventions [1] and automation framework practices:

- Canonical grammar rules defining valid feature syntax.
- Step definition patterns and implementation templates.
- Example behavior driven development feature files.
- Framework specific automation rules.
- Runtime user interface discovery outputs.
- Locator metadata extracted from the target system.

These documents collectively define the permissible generation space for the language model.

B. Deterministic Ranking Strategy

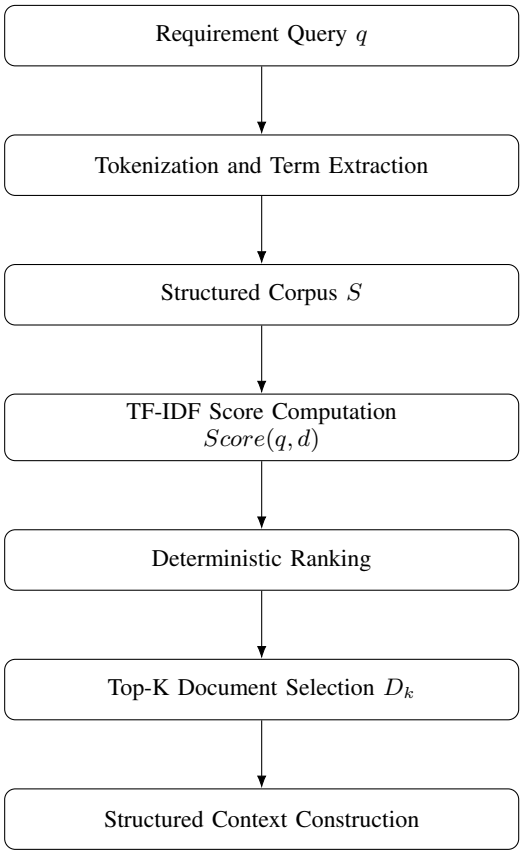

Fig. 2. Deterministic TF-IDF ranking and Top-K context selection pipeline.

Given a requirement derived query, documents are ranked using a term frequency inverse document frequency scoring

mechanism derived from classical information retrieval theory [14]. This lexical scoring strategy ensures deterministic behavior under a fixed corpus.

Unlike embedding similarity approaches [23], this ranking method does not rely on probabilistic similarity models or external vector databases. As a result, retrieval behavior remains fully reproducible.

C. Runtime User Interface Injection

A key component of the grounding mechanism is the integration of runtime user interface discovery outputs. Prior to generation, the system extracts visible interface elements and locator properties from the target application using automated web testing techniques [12]. These elements are injected into the retrieval corpus and ranked alongside static framework documents.

This design ensures that generated steps reference only verified interface elements. If required elements are not present in the discovery output, generation is constrained through refusal policies rather than hallucination, addressing reliability concerns identified in large language model research [5].

D. Structured Context Construction

Selected documents are formatted into structured prompt sections and inserted into the model input before inference, consistent with structured retrieval augmentation pipelines [7].

The final model input therefore consists of:

1) Normalized requirement intent
2) Retrieved framework rules and patterns
3) Runtime user interface discovery context
4) Explicit instruction constraints

This structured injection limits generation to domain verified patterns and reduces off specification outputs.

E. Grounded Determinism Property

Because both retrieval ranking and language model decoding follow deterministic formulations under transformer based autoregressive modeling [11], the overall grounding process behaves as a deterministic mapping. This property distinguishes the proposed system from stochastic retrieval and sampling based generation pipelines [7].

VIII. Guardrails and Governance Framework

To ensure structural correctness, reproducibility, and enterprise readiness, the proposed system incorporates a multi layer guardrail and governance framework aligned with trustworthy artificial intelligence principles [10].

A. Canonical Grammar Enforcement

Generated feature files must comply with predefined canonical grammar rules derived from behavior driven development specifications [1]. Grammar enforcement ensures syntactic consistency and prevents malformed test artifacts in alignment with established testing documentation standards [13].

B. Refusal Semantics

To prevent hallucinated elements, the system incorporates explicit refusal policies. This mechanism prioritizes correctness over completeness and addresses reliability challenges associated with large language model deployment [5].

C. Structural Validation Layer

All generated outputs are passed through a validation function before execution. Only validated artifacts proceed to the execution stage, consistent with structured testing validation standards [27].

D. Model Freeze and Configuration Control

The deployed system operates under a strict model freeze policy. This governance aligned configuration supports reproducibility and auditability requirements emphasized in trustworthy AI systems [10].

E. Logging and Auditability

Execution traces, inference metadata, and validation outcomes are recorded to support debugging and audit requirements, consistent with software testing documentation practices [13].

F. Safety Through Deterministic Design

Because retrieval, decoding, and validation follow deterministic transformer based inference principles [11], the overall system behaves as a controlled mapping from requirement input to execution output. This deterministic safety boundary enhances trust and simplifies reproducibility in enterprise deployments.

IX. Experimental Evaluation

This section evaluates the deterministic behavior, structural compliance, and execution validity of the proposed framework.

A. Experimental Setup

The system was evaluated in a controlled local environment using the fine tuned language model with fixed inference parameters. The training dataset consisted of approximately 2000 structured domain specific samples. The adapted model achieved approximately 85 percent token level accuracy on a held out validation set during fine tuning.

All inference experiments were conducted under deterministic decoding configuration with sampling disabled and greedy token selection enabled. Retrieval corpora and configuration parameters remained static across runs.

B. Deterministic Reproducibility Validation

To verify deterministic behavior, identical requirement inputs were processed multiple times under identical configuration settings. In all repeated trials, the generated outputs were identical. This confirms that the generation pipeline behaves as a deterministic mapping from input to output.

C. Grammar Compliance Evaluation

Generated feature files were validated against canonical grammar constraints and structural rules. The validation layer ensured correct feature structure, step formatting, and subject normalization. All accepted outputs satisfied predefined grammar requirements prior to execution.

D. Refusal Behavior Verification

Controlled test cases were constructed where essential user interface elements were intentionally absent from runtime discovery. In these scenarios, the system returned structured error responses rather than generating speculative steps. This demonstrates effective enforcement of refusal semantics and reduction of hallucinated outputs.

E. Execution Validation

Validated artifacts were executed within the integrated automation framework. The execution layer successfully processed structurally valid feature files without runtime structural failures. Error handling mechanisms correctly captured and categorized failures when they occurred.

F. Discussion of Results

The evaluation confirms that the proposed framework achieves deterministic reproducibility, structural compliance, and grounded generation. While the system does not claim universal accuracy across arbitrary domains, the integration of retrieval grounding, constraint enforcement, and deterministic inference significantly improves reliability compared to unconstrained stochastic generation pipelines.

X. Discussion

The experimental evaluation demonstrates that the proposed framework achieves deterministic reproducibility and structurally compliant generation under controlled inference conditions. By integrating retrieval grounding, grammar enforcement, and refusal semantics, the system reduces hallucinated outputs and improves alignment with domain specific automation rules. The deterministic decoding protocol further ensures that identical inputs produce identical outputs, which is essential for enterprise adoption and auditability.

The modular multi agent architecture contributes to traceability and maintainability. Each transformation stage is isolated, allowing targeted debugging and incremental enhancement. The integration of runtime user interface discovery into the retrieval corpus strengthens grounding and limits generation to verified interface elements.

Despite these strengths, several limitations remain. The fine tuning dataset consists of approximately 2000 samples, which may limit generalization across highly diverse domains. The evaluation focuses primarily on structural correctness and reproducibility rather than large scale benchmark comparison. Additionally, while deterministic decoding improves consistency, it may reduce generative flexibility in highly ambiguous requirement scenarios.

Future improvements can be explored in multiple directions. Expanding the training dataset with more diverse automation scenarios may improve domain coverage. Hybrid retrieval strategies that combine lexical ranking with lightweight semantic similarity could further enhance contextual relevance while preserving reproducibility. Adaptive validation mechanisms may also be introduced to provide graded confidence scores rather than binary acceptance or refusal. Furthermore, integration with continuous integration pipelines and large scale real world case studies would strengthen empirical validation.

Overall, the proposed deterministic multi agent framework establishes a structured and reproducible foundation for artificial intelligence assisted test automation, while offering clear avenues for future refinement and scalability.

XI. Conclusion

This paper presented OrionTest, a deterministic multi agent architecture for requirement driven behavior driven development test automation. The proposed framework integrates structured retrieval grounding, grammar constrained generation, and controlled inference to address limitations of stochastic language model based automation systems.

By combining domain specific fine tuning, deterministic decoding, runtime user interface discovery, and multi layer validation, the system provides reproducible and structurally compliant test artifact generation. The modular agent design enhances traceability and maintainability, while the governance framework enforces auditability and execution level validation.

Experimental validation confirms deterministic reproducibility, effective grammar enforcement, and grounded generation behavior under fixed configuration settings. Although further large scale empirical studies are required, the proposed architecture establishes a structured foundation for enterprise ready artificial intelligence assisted test automation.

Future work will focus on expanding domain coverage, improving contextual retrieval strategies, and evaluating scalability across diverse real world software systems.

References

[1] D. North, “Introducing BDD,” Better Software Magazine, vol. 7, no. 6, pp. 36–39, 2006.

[2] K. Beck, Test-driven development: By example. Boston, MA: Addison-Wesley, 2003.

[3] T. Xie and D. Notkin, “Tool-assisted unit test generation and selection based on operational abstractions,” Autom. Softw. Eng., vol. 15, no. 3, pp. 257–286, 2008.

[4] T. Chen, S. Liu, and Y. Chen, “Automated test case generation using natural language processing,” IEEE Access, vol. 8, pp. 123456–123468, 2020.

[5] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, and A. Amodei, “Language models are few-shot learners,” in Proc. NeurIPS, 2020, pp. 1877–1901.

[6] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in Proc. ICLR, 2022.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, and S. Riedel, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. NeurIPS, 2020.

[8] K. Guu, T. Hashimoto, Y. Oren, and P. Liang, "REALM: Retrieval-augmented language model pre-training," in Proc. ICML, 2020.

[9] M. Wooldridge, An introduction to multiagent systems, 2nd ed. Chichester, U.K.: Wiley, 2009.

[10] L. Floridi and J. Cowls, "A unified framework of five principles for AI in society," Harvard Data Science Review, vol. 1, no. 1, 2019.

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, and I. Polosukhin, "Attention is all you need," in Proc. NeurIPS, 2017, pp. 5998–6008.

[12] S. Artzi, J. Dolby, S. Jensen, A. Møller, and F. Tip, "A framework for automated testing of JavaScript web applications," in Proc. ICSE, 2011, pp. 571–580.

[13] IEEE, "IEEE standard for software and system test documentation," IEEE Std 829-2008, 2008.

[14] M. G. Jorgensen, Software testing: A craftsman's approach, 4th ed. Boca Raton, FL: CRC Press, 2013.

[15] G. Myers, C. Sandler, and T. Badgett, The art of software testing, 3rd ed. Hoboken, NJ: Wiley, 2011.

[16] A. Bertolino, "Software testing research: Achievements, challenges, dreams," in Proc. Future of Software Engineering, 2007, pp. 85–103.

[17] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: A survey," Software Testing, Verification and Reliability, vol. 22, no. 2, pp. 67–120, 2012.

[18] M. Pradel and K. Sen, "DeepTest: Automated testing of deep neural network driven autonomous cars," in Proc. ICSE, 2018, pp. 303–314.

[19] Y. Liu, Z. Chen, and C. Xu, "Automated generation of test cases from natural language requirements: A systematic literature review," Information and Software Technology, vol. 121, 2020.

[20] OpenAI, "GPT-4 technical report," 2023.

[21] A. Creswell, M. Shanahan, and I. Higgins, "Selection-inference: Exploiting large language models for interpretable logical reasoning," arXiv preprint arXiv:2205.09712, 2022.

[22] S. Karpukhin, V. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, et al., "Dense passage retrieval for open-domain question answering," in Proc. EMNLP, 2020.

[23] N. Shinn, B. Labash, and A. Goyal, "Reflexion: Language agents with verbal reinforcement learning," in Proc. NeurIPS, 2023.

[24] J. Yao, S. Yu, and Y. Zhao, "Tree of thoughts: Deliberate reasoning with language models," in Proc. NeurIPS, 2023.

[25] T. Schick and H. Schütze, "Self-training with noisy student improves natural language understanding," in Proc. ACL, 2021.

[26] ISO/IEC, "ISO/IEC 29119 software testing standards," 2013.

[27] Microsoft, "Playwright documentation," Microsoft Corporation, 2023.

[28] R. Fielding and R. Taylor, "Principled design of the modern Web architecture," ACM Transactions on Internet Technology, vol. 2, no. 2, pp. 115–150, 2002.